

华中科技大学

课程实验报告

课程名称：计算机系统基础

专业班级 2401

学号 U202490042

姓名 马俊豪

指导教师 你的指导教师

报告日期 2026 年 X 月 X 日

计算机科学与技术学院

目 录

实验 1: 数据表示和等效运算	1
1.1 实验概述	1
1.2 实验内容	1
1.3 实验设计	4
1.4 实验过程	4
1.4.1 环境准备	4
1.4.2 具体实施步骤	4
1.5 实验结果	5
1.5.1 结果展示	5
1.5.2 结果分析	5
1.6 实验小结	6
1.6.1 理论与方法总结	6
1.6.2 结果与收获总结	6
实验 2: 汇编和 C 的混合编程及优化	7
2.1 实验概述	7
2.2 实验内容	7
2.3 实验设计	8
2.3.1 解题思路分析	8
2.3.2 技术与方法	8
2.4 实验过程	8
2.4.1 环境准备	8
2.4.2 具体实施步骤	8
2.5 实验结果	8
2.5.1 结果展示	8
2.5.2 结果分析	8
2.6 实验小结	8
2.6.1 理论与方法总结	8
2.6.2 结果与收获总结	9
实验 3: 机器级语言理解 (二进制炸弹拆除)	10

华中科技大学课程实验报告

3.1	实验概述	10
3.2	实验内容	10
3.3	实验设计	11
3.3.1	解题思路分析	11
3.3.2	技术与方法	11
3.4	实验过程	11
3.4.1	环境准备	11
3.4.2	具体实施步骤	11
3.5	实验结果	11
3.5.1	结果展示	11
3.5.2	结果分析	12
3.6	实验小结	12
3.6.1	理论与方法总结	12
3.6.2	结果与收获总结	12
实验 4: 缓冲区溢出攻击		13
4.1	实验概述	13
4.2	实验内容	13
4.3	实验设计	14
4.3.1	解题思路分析	14
4.3.2	技术与方法	14
4.4	实验过程	14
4.4.1	环境准备	14
4.4.2	具体实施步骤	14
4.5	实验结果	14
4.5.1	结果展示	14
4.5.2	结果分析	14
4.6	实验小结	14
4.6.1	理论与方法总结	14
4.6.2	结果与收获总结	15
实验 5: ELF 文件与程序链接		16
5.1	实验概述	16

华中科技大学课程实验报告

5.2	实验内容	16
5.3	实验设计	19
5.3.1	解题思路分析	19
5.3.2	技术与方法	19
5.4	实验过程	19
5.4.1	环境准备	19
5.4.2	具体实施步骤	19
5.5	实验结果	19
5.5.1	结果展示	19
5.5.2	结果分析	19
5.6	实验小结	19
5.6.1	理论与方法总结	19
5.6.2	结果与收获总结	20
6	实验总结	21
A	附录：实验源代码	22
A.1	实验目录结构	22
A.2	核心程序源代码	22
	参考文献	23

实验 1：数据表示和等效运算

1.1 实验概述

1. 熟练掌握程序开发平台 (VS2019/Vs2022/Vs2023) 的基本用法，包括程序的编译、链接和调试；
2. 熟悉数据的表示形式；
3. 熟悉地址的计算方法、地址的内存转换；
4. 用常见的按位操作（移位、按位与/或/非/异或）等实现运算表达式的等效运算。

1.2 实验内容

任务 1 数据存放的压缩与解压编程

定义了结构 `student`，以及结构数组变量 `old_s[N]`, `new_s[N]`; ($N=5$)

```
1 struct student {  
2     char   name[8];  
3     short  age;  
4     float  score;  
5     char   remark[200]; // 备注信息  
6 };
```

编写程序，输入 N 个学生的信息到结构数组 `old_s` 中。将 `old_s[N]` 中的所有信息依次紧凑 (压缩) 存放到一个字符数组 `message` 中，然后从 `message` 解压缩到结构数组 `new_s[N]` 中。打印压缩前 (`old_s`)、解压后 (`new_s`) 的结果，以及压缩前、压缩后存放数据的长度。

要求：

1. 输入的第 0 个人姓名 (`name`) 为自己的名字，分数为学号的最后两位；
2. 编写指定接口的函数完成数据压缩

压缩函数有两个：

```
1 int pack_student_bytebybyte(student* s, int sno, char *buf);  
2 int pack_student_whole(student* s, int sno, char *buf);
```

s 为待压缩数组的起始地址；sno 为压缩人数；buf 为压缩存储区的首地址；两个函数的返回均是调用函数压缩后的字节数。pack_student_bytebybyte 要求一个字节一个字节的向 buf 中写数据；pack_student_whole 要求对 short、float 字段都只能用一条语句整体写入，用 strcpy 实现串的写入。

3. 使用指定方式调用压缩函数

old_s 数组的前 N1 (N1=2) 个记录压缩调用 pack_student_bytebybyte 完成；后 N2 (N2=3) 个记录压缩调用 pack_student_whole，两种压缩函数都只调用 1 次。

4. 使用指定的函数完成数据的解压

解压函数的格式：

```
1 int restore_student(char *buf, int len, student* s);
```

buf 为压缩区域存储区的首地址；len 为 buf 中存放数据的长度；s 为存放解压数据的结构数组的起始地址；返回解压的人数。解压时不允许使用函数接口之外的信息（即不允许定义其他全局变量）

5. 仿照调试时看到的内存数据，以十六进制的形式，输出 message 的前 20 个字节的內容，并与调试时在内存窗口观察到的 message 的前 20 个字节比较是否一致。
6. 对于第 0 个学生的 score，根据浮点数的编码规则指出其个部分的编码，并与观察到的内存表示比较，验证是否一致。
7. 指出结构数组中个元素的存放规律，指出字符串数组、short 类型的数、float 型的数的存放规律。

任务 2 编写位运算程序

按照要求完成给定的功能，并自动判断程序的运行结果是否正确。（从逻辑电路与门、或门、非门等等角度，实现 CPU 的常见功能。所谓自动判断，即用简单的方式实现指定功能，并判断两个函数的输出是否相同。）

1. int absVal(int x); 返回 x 的绝对值

仅使用!、~、&、^、|、+、«、»，运算次数不超过 10 次

判断函数: `int absVal_standard(int x) { return (x < 0) ? -x : x; }`

2. `int negate(int x)`; 不使用负号, 实现 $-x$
判断函数: `int negate_standard(int x) { return -x; }`
3. `int bitAnd(int x, int y)`; 仅使用 $\sim$ 和 $|$, 实现 $\&$
判断函数: `int bitAnd_standard(int x, int y) { return x & y; }`
4. `int bitOr(int x, int y)`; 仅使用 $\sim$ 和 $\&$, 实现 $|$
5. `int bitXor(int x, int y)`; 仅使用 $\sim$ 和 $\&$, 实现 $\wedge$
6. `int isTmax(int x)`; 判断 x 是否为最大的正整数 ($7FFFFFFF$),
只能使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$
7. `int bitCount(int x)`; 统计 x 的二进制表示中 1 的个数
只能使用, $! \sim \& ^ | + \ll \gg$, 运算次数不超过 40 次
8. `int bitMask(int highbit, int lowbit)`; 产生从 `lowbit` 到 `highbit` 全为 1, 其他位为 0 的数。例如 `bitMask(5,3) = 0x38`; 要求只使用 $! \sim \& ^ | + \ll \gg$; 运算次数不超过 16 次。
9. `int addOK(int x, int y)`; 当 $x+y$ 会产生溢出时返回 1, 否则返回 0
仅使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 20 次
10. `int byteSwap(int x, int n, int m)`; 将 x 的第 n 个字节与第 m 个字节交换, 返回交换后的结果。 n 、 m 的取值在 0~3 之间。
例: `byteSwap(0x12345678, 1, 3) = 0x56341278`
`byteSwap(0xDEADBEEF, 0, 2) = 0xDEEFBEAD`
仅使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 25 次
11. `int bang(int x)` 当 $x=0$ 时, 返回 1; 其他情况返回 0。实现逻辑非 (!)
例: `bang(3) = 0; bang(0) = 1;`
仅使用 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 12 次
提示: 只有当 $x=0$ 时, x 与 $-x$ 的最高二进制位会同时为 0。
12. `int bitParity(int x)` 当 x 有奇数个二进制位 0, 返回 1; 否则返回 0
例: `bitParity(5) = 0; bitParity(7) = 1;`
仅使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 20 次。
提示: 只有当 x 的高字与低字的对应位 (第 0 位对应第 16 位, 第 1 位对应第 17 位, 依次类推) 同时为 1, 则出现了成对的二进制位 1, 此时, 可以将对应的二进制位置为 0, 不会影响二进制位 1 的个数的奇偶性判断。

1.3 实验设计

本实验设计分为两个部分。

对于任务 1，采用顺序存储的思想，将结构体中的字符串字段按字符序列加结束符存入缓冲区，将 short 和 float 字段按其字节表示写入缓冲区，从而实现去除结构体对齐空隙后的紧凑存储；解压时按照相同顺序解析缓冲区，恢复各字段内容。该设计重点体现了字符串存储方式、数值类型内存表示和结构体内存布局的理解。

对于任务 2，采用补码运算和布尔代数等效变换的方法，将绝对值、按位逻辑、溢出判断、位计数、字节交换等功能转化为移位、掩码和按位逻辑运算实现，并通过与标准函数对比的方式验证结果正确性。整体设计体现了从底层数据表示出发分析问题和构造算法的思路。

Algorithm 1 算法示例

Input: 输入数据或系统状态

Output: 程序输出或分析结果

1: 在此填写算法步骤或删除本示例

1.4 实验过程

1.4.1 环境准备

- 操作系统：例如 *Windows 11 / Ubuntu 22.04*
- 编译器或解释器：例如 *GCC 13 / Clang 17 / Python 3.12*
- 开发工具：例如 *VS Code / Visual Studio / GDB / Readelf / Objdump*
- 其他依赖：例如 *Make / CMake / LaTeX / Docker*

1.4.2 具体实施步骤

1. 步骤 1: 准备实验文件、配置运行环境，并明确实验输入与目标。
2. 步骤 2: 根据设计思路编写、修改或分析程序，记录关键操作。
3. 步骤 3: 编译、运行、调试程序，并保存中间结果、截图或日志。
4. 步骤 4: 根据实验现象继续修正、优化和验证，直至满足实验要求。

1 # 在此填写实验中使用的命令行操作

```
2 gcc -Wall -Wextra -O0 -g main.c -o main
3 ./main
```

Listing 1 命令行操作示例

```
1 /* 在此填写核心代码片段，保留最能体现思路的部分 */
2 int main(void) {
3     return 0;
4 }
```

Listing 2 核心代码示例

1.5 实验结果

1.5.1 结果展示

表 1-1 测试结果记录表

测试编号	输入或场景	预期结果	实际结果
Test 1	填写测试输入	填写预期结果	填写实际结果
Test 2	填写测试输入	填写预期结果	填写实际结果
Test 3	填写测试输入	填写预期结果	填写实际结果

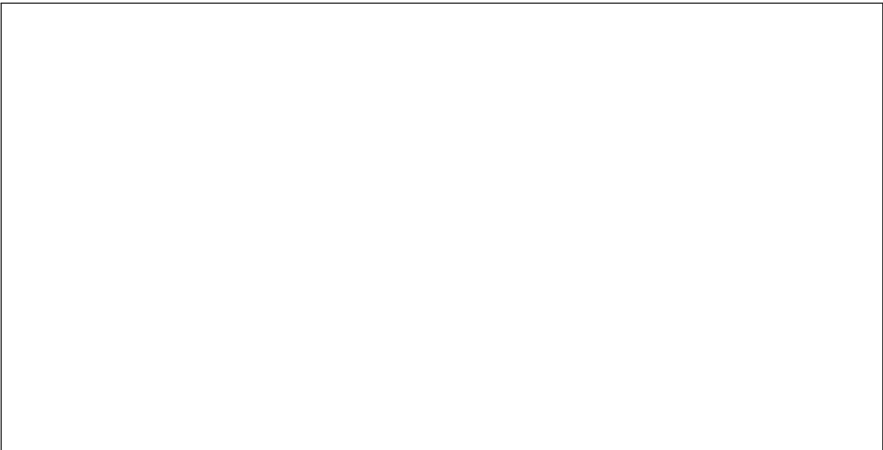


图 1-1 实验截图占位图

1.5.2 结果分析

结合实验结果进行必要分析，说明实验是否达到预期，并分析错误原因、边界情况、性能表现或与理论值的偏差。

1.6 实验小结

1.6.1 理论与方法总结

对本次实验使用到的理论、技术、方法和关键实现思路进行总结。

1.6.2 结果与收获总结

总结本次实验的结果、收获、存在的问题，以及后续可改进的方向。

实验 2：汇编和 C 的混合编程及优化

2.1 实验概述

1. 熟练掌握程序开发平台 (VS2019/Vs2022/Vs2023) 下 C 和汇编语言的混合编程方法；
2. 熟悉掌握常用机器指令的使用方法；
3. 掌握代码优化的基本方法，提高程序运行速度。

2.2 实验内容

任务 1 用 C 语言编写一个学生成绩管理程序，具有平均成绩计算、按平均成绩从高到低排序、显示学生成绩的功能

定义了结构 `student`，设有 `N` 个学生

```
1 struct student {  
2     char name[8];  
3     char sid[11]; // 如 U202315123  
4     short scores[8]; // 8 门课的分  
5     short average; // 平均分  
6 };
```

要求：

1. 第 0 个人姓名 (`name`) 为自己的名字，`sid` 为自己的学号；
2. 对于函数“计算平均成绩”、“按平均成绩从高到低排序”分别计时，显示两个函数的时间开销；
3. 显示排序前、排序后的学生信息（姓名、学号、8 门课的分、平均分），一个学生一行。
4. 对“计算平均成绩”、“按平均成绩排序”函数的 Debug 版、Release 版的运行效率进行对比，比较两个版本下的反汇编代码的差异，分析程序执行速度提高的原因。

任务 2 用汇编语言编写函数“计算平均成绩”，代替原来用 C 语言编写的函数

任务 3 对用汇编语言编写的“计算平均成绩”函数进行优化

任务 4 对排序函数进行算法优化

2.3 实验设计

2.3.1 解题思路分析

给出问题分析、总体求解思路，以及为什么采用当前方案来完成实验任务。

2.3.2 技术与方法

说明拟采用的技术、方法、数据结构、关键算法、模块划分或实现策略。

2.4 实验过程

2.4.1 环境准备

说明本实验使用的软硬件环境、工具链和依赖配置。

2.4.2 具体实施步骤

按步骤详细描述实验的具体操作过程，可结合命令、代码修改、调试记录和关键截图展开说明。

2.5 实验结果

2.5.1 结果展示

展示实验输出、测试结果、运行截图或关键现象。

2.5.2 结果分析

对实验结果进行必要分析，说明实验是否达到预期，并分析原因。

2.6 实验小结

2.6.1 理论与方法总结

对本次实验使用到的理论、技术、方法和关键实现思路进行总结。

2.6.2 结果与收获总结

总结本次实验的结果、收获、存在的问题，以及后续可改进的方向。

实验 3：机器级语言理解 (二进制炸弹拆除)

3.1 实验概述

1. 熟练掌握程序开发平台 (VS2019/Vs2022/Vs2023) 下 C 和汇编语言的混合编程方法；
2. 熟悉掌握常用机器指令的使用方法；
3. 掌握代码优化的基本方法，提高程序运行速度。

3.2 实验内容

操作说明

整个程序包含有：bomb.c support.c phases.c support.h。在老师手上有 phases.c，但同学们没有该源程序，而只有 phases.o。

提供给学生的包：bomb.c support.c phases.o support.h

由学生生成执行程序：

```
1 gcc -g -o bomb -D U* bomb.c support.c phases.o
```

其中 * 为学号的最后一位。例如：学号为 U202415001，则使用命令

```
1 gcc -g -o bomb -D U1 bomb.c support.c phases.o
```

运行程序：

```
1 ./bomb
```

下面给出了三关通过的效果。

```
1 root@LAPTOP-CJLSTBTI:/home/binary_bomb#
2 root@LAPTOP-CJLSTBTI:/home/binary_bomb# gcc -c -D PROBLEM phases.c -o phases.o
3 root@LAPTOP-CJLSTBTI:/home/binary_bomb# gcc -g -o bomb -D U3 bomb.c support.c phases.o
4 root@LAPTOP-CJLSTBTI:/home/binary_bomb# ./bomb
5 Input your Student ID :
6 U202215123
7 welcome U202215123
8 You have 6 bombs to defuse!
9 Gate 1 : input a string that meets the requirements.
```

```
10 Computer System Foundation.
11 Phase 1 passed!
12 Gate 2 : input six intergers that meets the requirements.
13 3 2 5 7 12 19
14 Phase 2 passed!
15 Gate 3 :
16 input 2 intergers.
17 1 304
```

提醒:

1. 一定要输入自己的学号，运行结果中会显示学号的。
2. 在生成程序时，一定要带编译开关 `-D U*`；* 为学号的最后一位数字。

3.3 实验设计

3.3.1 解题思路分析

给出问题分析、总体求解思路，以及为什么采用当前方案来完成实验任务。

3.3.2 技术与方法

说明拟采用的技术、方法、数据结构、关键算法、模块划分或实现策略。

3.4 实验过程

3.4.1 环境准备

说明本实验使用的软硬件环境、工具链和依赖配置。

3.4.2 具体实施步骤

按步骤详细描述实验的具体操作过程，可结合命令、代码修改、调试记录和关键截图展开说明。

3.5 实验结果

3.5.1 结果展示

展示实验输出、测试结果、运行截图或关键现象。

3.5.2 结果分析

对实验结果进行必要分析，说明实验是否达到预期，并分析原因。

3.6 实验小结

3.6.1 理论与方法总结

对本次实验使用到的理论、技术、方法和关键实现思路进行总结。

3.6.2 结果与收获总结

总结本次实验的结果、收获、存在的问题，以及后续可改进的方向。

实验 4：缓冲区溢出攻击

4.1 实验概述

通过分析一个程序（称为“缓冲区炸弹”）的构成和运行逻辑，加深对理论课中关于程序的机器级表示、函数调用规则、栈结构等方面知识点的理解，增强反汇编、跟踪、分析、调试等能力，加深对缓冲区溢出攻击原理、方法与防范等方面知识的理解和掌握；

实验环境：Ubuntu, GCC, GDB 等

4.2 实验内容

任务缓冲区溢出攻击

程序运行过程中，需要输入特定的字符串，使得程序达到期望的运行效果。

对一个可执行程序“bufbomb”实施一系列缓冲区溢出攻击 (buffer overflow attacks)，也就是设法通过造成缓冲区溢出来改变该程序的运行内存映像 (例如将专门设计的字节序列插入到栈中特定内存位置) 和行为，以实现实验预定的目标。bufbomb 目标程序在运行时使用函数 `getbuf` 读入一个字符串。根据不同的任务，学生生成相应的攻击字符串。

实验中需要针对目标可执行程序 `bufbomb`，分别完成多个难度递增的缓冲区溢出攻击 (完成的顺序没有固定要求)。按从易到难的顺序，这些难度级分别命名为 `smoke (level 1)`、`fizz (level 2)`。

1、第 1 级 smoke

正常情况下，`getbuf` 函数运行结束，执行最后的 `ret` 指令时，将取出保存于栈帧中的返回（断点）地址并跳转至它继续执行（`test` 函数中调用 `getbuf` 处）。要求将返回地址的值改为本级别实验的目标 `smoke` 函数的首条指令的地址，`getbuf` 函数返回时，跳转到 `smoke` 函数执行，即达到了实验的目标。

2、第 2 级 fizz

要求 `getbuf` 函数运行结束后，转到 `fizz` 函数处执行。与 `smoke` 的差别是，`fizz` 函数有一个参数。`fizz` 函数中比较了参数 `val` 与全局变量 `cookie` 的值，只有两者相同（要正确打印 `val`）才能达到目标。

4.3 实验设计

4.3.1 解题思路分析

给出问题分析、总体求解思路，以及为什么采用当前方案来完成实验任务。

4.3.2 技术与方法

说明拟采用的技术、方法、数据结构、关键算法、模块划分或实现策略。

4.4 实验过程

4.4.1 环境准备

说明本实验使用的软硬件环境、工具链和依赖配置。

4.4.2 具体实施步骤

按步骤详细描述实验的具体操作过程，可结合命令、代码修改、调试记录和关键截图展开说明。

4.5 实验结果

4.5.1 结果展示

展示实验输出、测试结果、运行截图或关键现象。

4.5.2 结果分析

对实验结果进行必要分析，说明实验是否达到预期，并分析原因。

4.6 实验小结

4.6.1 理论与方法总结

对本次实验使用到的理论、技术、方法和关键实现思路进行总结。

4.6.2 结果与收获总结

总结本次实验的结果、收获、存在的问题，以及后续可改进的方向。

实验 5：ELF 文件与程序链接

5.1 实验概述

通过对给定的源程序进行编译，阅读和分析编译生成的汇编语言程序、可重定位的目标文件、可执行目标文件，加深对编译过程、目标文件主要节（节中的内容、节与节的关系）、目标文件的生成、以及链接的理论知识的理解。

1. 区分.c 源文件、.i 预处理文件、.s 汇编文件、.o 目标文件、可执行 ELF 文件差异；
2. 掌握可重定位目标文件中，文件头、节头表、.text、.data、.bss、.rodata、.symtab、.rel.text、.rel.data、.strtab 等节中的核心组成成份；
3. 掌握一个程序中哪些标识符是符号，掌握符号表中各符号所包含的信息，包括符号的类型、符号定义的位置（节）、符号的值、符号的大小、绑定属性，以及与 strtab 的关系；
4. 掌握重定位节 .rel.text、.rel.data 的各条目的信息组成；对于汇编程序，能否确定哪些地方需要重定位；
5. 对比可重定位目标文件与可执行目标文件的差异；
6. 掌握符号重定位原理、未定义符号地址填充过程；
7. 能够找出静态链接、动态链接生成可执行程序差异；
8. 理解地址无关代码 PIC、动态库共享内存机制。

实验环境：Ubuntu

工具：GCC、GDB、readelf、hexdump、objdump 等。

5.2 实验内容

给定了源程序 main.c、sub.c。学生要按要求对 main.c 中的两处进行修改：char init_gstr1[12]="U202415001" // 请换成自己的学号

```
1 __asm__ __volatile__(".rept # \n nop \n .endr \n");
```

// 将 # 换成学号的最后 1 位

在编译时，注意带上编译开关 -D U*，* 为学号的最后一位

任务 1 从 C 程序到汇编语言程序的实践

使用

```
1 gcc -S -fverbose-asm -D U* main.c -o main.s
```

1. 以具体的示例（如 `init_gstr1`, `uninit_gstr2`），介绍全局变量汇编程序中的表示方式，以及在什么情况下会放在 `.data` 段，何时又会放在 `.bss` 段。
2. 以函数调用语句为例，给出 C 语句与汇编语句的对应关系。
3. 以具体的 C 语句及对应的汇编语句为例，说明全局变量与局部变量在汇编程序中的表达形式的差异。

任务 2 可重定位目标文件解析

```
1 gcc -c -g -D U* main.c -o main.o
```

1、节头表

```
1 readelf -S -W main.o
```

描述 `main.o` 中主要有哪些节，各节包含哪些信息（各列的含义）；

2、与全局变量及静态局部变量相关的节

(1)

```
1 readelf -x .data main.o
```

或

```
1 objcopy -O binary --only-section=.data main.o /dev/stdout | hexdump -C
```

显示数据节 (`.data`) 的内容，指出显示的内容与定义变量时的初始化值的对应关系；

(2)

```
1 readelf -s main.o | awk '$7==3 || NR==3'
```

显示符号表中在 `.data` 节定义了哪些符号；指出符号的位置 (`value`) 与 `.data` 中的位置关系；

(3) 使用 `readelf` 显示未初始化数据节 (`.bss`) 中包含的符号，研究为什么这些符号会放在 `.bss` 节；

(4) 展示 `.data.rel.local`、`.rela.data.rel.local`、`.data.rel.rela.data.rel` 中包含的内容，说明什么样的全局变量会出现在相应的节中；

(5) 显示 `.strtab` 节，说明在 `.strtab` 中出现了哪些种类的变量的名称。

(6) 描述一个全局变量定义语句，在可重定位目标文件的哪些节各会生成哪些信息。

3、代码节及代码节的重定位节

1. 显示.text 节以及.rela.text 节的内容，描述其对应关系（重定位节包含的核心信息内容）；
2. 显示符号表中在.text 节定义了哪些符号。

4、符号表节

1. 显示符号表，指出符号表中各列的含义；
2. 指出有哪些符号是未定义的，说明为什么这些符号未定义。

5、只读节

1. 显示.rodata 中的内容，描述程序中有哪些内容会放到.rodata 节中；
2. 以 printf 的函数调用示例（例如：

```
1 printf("display init_gstr1 : %s \n", init_gstr1);
```

说明格式串的首地址是如何定位的。

任务 3 可执行目标文件解析

```
1 gcc -g -D U* main.c sub.c -o main_sub
2 gcc -g -D U* sub.c main.c -o sub_main
```

1. 比较两个文件（main_sub、sub_main）中，全局变量的排列顺序，函数的排列顺序，探讨链接时符号地址确定规律；
2. 显示可执行文件的符号表，指出符号表中各符号的地址与可重定位目标文件中各符号地址的差异；
3. 以 main 函数的汇编语言代码为例，说明可执行文件中机器语言语句与可重定位目标文件中的机器语言语句（包括机器码，反汇编语句）有什么差别。
4. 解释符号定义、符号绑定、符号引用、重定位的基本概念。
5. 描述生成可执行程序的基本过程。

任务 4 静态链接生成执行程序

比较静态链接与动态链接生成的可执行程序的差异。

任务 5 研究动态链接库中的函数 (如 printf) 的定位方法

5.3 实验设计

5.3.1 解题思路分析

给出问题分析、总体求解思路，以及为什么采用当前方案来完成实验任务。

5.3.2 技术与方法

说明拟采用的技术、方法、数据结构、关键算法、模块划分或实现策略。

5.4 实验过程

5.4.1 环境准备

说明本实验使用的软硬件环境、工具链和依赖配置。

5.4.2 具体实施步骤

按步骤详细描述实验的具体操作过程，可结合命令、代码修改、调试记录和关键截图展开说明。

5.5 实验结果

5.5.1 结果展示

展示实验输出、测试结果、运行截图或关键现象。

5.5.2 结果分析

对实验结果进行必要分析，说明实验是否达到预期，并分析原因。

5.6 实验小结

5.6.1 理论与方法总结

对本次实验使用到的理论、技术、方法和关键实现思路进行总结。

5.6.2 结果与收获总结

总结本次实验的结果、收获、存在的问题，以及后续可改进的方向。

6 实验总结

全面总结本次实验的整体成果，归纳实验中使用到的理论、技术、方法与最终结果，并结合实际过程描述通过实验获得的能力提升、问题认识和经验收获。建议本节正文不少于 400 字。

A 附录：实验源代码

A.1 实验目录结构

```
1 LAB_1/  
2   main.c  
3   helper.c  
4   helper.h  
5   Makefile  
6   README.md
```

Listing 3 实验目录结构示例

A.2 核心程序源代码

```
1 /* 在附录中放置完整源码或关键源码 */  
2 int add(int left, int right) {  
3     return left + right;  
4 }
```

Listing 4 附录代码示例

参考文献

- [1] BRYANT R E, O'HALLARON D R. Computer Systems: A Programmer's Perspective[M]. 3rd ed. Boston : Pearson, 2016.
- [2] GNU Project. Debugging with GDB[EB/OL]. 2025 [2026-04-27].
<https://sourceware.org/gdb/current/onlinedocs/gdb/>.
- [3] 计算机系统基础实验指导书 [K]. 2026.